

Java Spring Boot Development Standards and Best Practices

1. Purpose

This document defines the engineering standards for developing Java Spring Boot applications to ensure:

- Maintainable and readable code
 - Consistent coding practices across teams
 - Scalable and secure applications
 - Reduced technical debt
 - Faster onboarding and development
 - Improved reliability and operational excellence
-

2. General Coding Standards

2.1 Follow Clean Code Principles

Guidelines

- Code should be easy to read and understand.
- Write code for humans first, computers second.
- Prefer readability over compactness.
- Minimize cognitive complexity.
- Keep implementation simple.

Recommended

```
public Customer findCustomerById(Long customerId) {  
    return customerRepository.findById(customerId)  
        .orElseThrow(() ->  
            new CustomerNotFoundException(customerId));  
}
```

Avoid

```
public Customer f(Long id){  
    return repo.findById(id).get();  
}
```

2.2 Keep Methods Small

Guidelines

- Method should perform only one task.
- Ideal method length: 10-20 lines.
- Maximum recommended length: 30 lines.

Bad:

```
public void processOrder() {  
    validate();  
    calculate();  
    updateInventory();  
    sendMail();  
    createAudit();  
    updateCRM();  
}
```

Good:

```
public void processOrder() {  
    validateOrder();  
    calculatePricing();  
    updateInventory();  
    notifyCustomer();  
}
```

2.3 Avoid Magic Numbers and Strings

Bad:

```
if (retryCount > 3)
```

Good:

```
private static final int MAX_RETRY_COUNT = 3;  
  
if (retryCount > MAX_RETRY_COUNT)
```

3. Coding Conventions and Naming Standards

3.1 Package Naming

Use lowercase.

```
com.company.project
com.company.project.controller
com.company.project.service
com.company.project.repository
```

Avoid:

```
com.Company.Project
```

3.2 Class Naming

Type	Convention	Example
Controller	Suffix Controller	UserController
Service	Suffix Service	UserService
Repository	Suffix Repository	UserRepository
DTO	Suffix Request/Response	UserRequest
Entity	Business Name	User
Exception	Suffix Exception	UserNotFoundException
Utility	Suffix Util	DateUtil
Configuration	Suffix Config	SecurityConfig

3.3 Method Naming

Method names must:

- Start with a verb.
- Clearly indicate behavior.

Examples:

```
createUser()  
findUserById()  
deleteCustomer()  
calculateDiscount()  
sendNotification()
```

Avoid:

```
doWork()  
process()  
execute()
```

3.4 Variable Naming

Good

```
employeeCount  
totalAmount  
customerAddress
```

Avoid

```
a  
b  
x1  
temp
```

3.5 Boolean Naming

Prefix boolean variables with:

```
is  
has  
can  
should
```

Examples:

```
isActive  
hasPermission  
canDelete  
shouldRetry
```

3.6 Constants

Always use:

```
public static final
```

Naming:

```
MAX_RETRY_COUNT  
DEFAULT_TIMEOUT_SECONDS
```

4. Formatting Standards

Indentation

- Use 4 spaces.
- Never use tabs.

Line Length

Recommended:

```
120 characters maximum
```

Braces

Always use braces.

Bad:

```
if (condition)
    execute();
```

Good:

```
if (condition) {
    execute();
}
```

One Statement Per Line

Bad:

```
int a = 1; int b = 2;
```

Good:

```
int a = 1;
int b = 2;
```

5. Comments and Documentation

Rules

Code should be self-explanatory.

Use comments only when necessary.

Avoid:

```
// increment count
count++;
```

Use comments for:

- Business rules

- Complex algorithms
 - Workarounds
 - Assumptions
-

JavaDoc

Mandatory for:

- Public APIs
- Shared libraries
- Utility classes

Example:

```
/**
 * Retrieves customer details by identifier.
 *
 * @param customerId customer identifier
 * @return customer details
 */
```

6. DRY Principle

(Don't Repeat Yourself)

Guidelines

- Avoid duplicate code.
- Extract reusable methods.
- Create common utility classes.
- Use inheritance/composition appropriately.

Duplicate business logic should be centralized.

7. SOLID Principles

Single Responsibility Principle

One class = One responsibility.

Bad:

```
UserService  
  validates  
  persists  
  sends email  
  creates reports
```

Good:

```
UserValidator  
UserService  
EmailService  
ReportService
```

Open Closed Principle

Classes should be extensible without modification.

Use:

- Interfaces
- Strategy Pattern

Liskov Substitution Principle

Derived classes should be replaceable.

Interface Segregation Principle

Create focused interfaces.

Bad:

```
interface Worker {  
  work();  
  eat();
```



```
        sleep();  
    }
```

Dependency Inversion Principle

Depend on abstractions.

Always use constructor injection.

Good:

```
@RequiredArgsConstructor  
@Service  
public class UserService {  
  
    private final NotificationService notificationService;  
}
```

Avoid:

```
@Autowired  
private NotificationService service;
```

8. Spring Boot Layered Architecture

Recommended structure:

```
controller  
service  
service.impl  
repository  
entity  
dto  
mapper  
config  
exception  
util  
constant  
validator
```

client
security

Controller Layer

Responsibilities:

- Accept requests
- Validate requests
- Return responses

Should NOT:

- Access repository directly
 - Contain business logic
-

Service Layer

Responsibilities:

- Business logic
 - Transactions
 - Orchestration
-

Repository Layer

Responsibilities:

- Database operations only

No business logic.

9. DTO and Entity Best Practices

Never expose entities directly.

Bad:

```
@GetMapping
public User getUser()
```

Good:

```
@GetMapping
public UserResponse getUser()
```

Benefits:

- API decoupling
- Better security
- Easier versioning

10. Exception Handling

Use Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {
}
```

Handle:

- ValidationException
- BusinessException
- ResourceNotFoundException
- DataAccessException

Create Domain Exceptions

Examples:

```
UserNotFoundException
OrderProcessingException
PaymentFailedException
```

Never

```
catch (Exception ex) {  
}
```

Never swallow exceptions.

11. Logging Standards

Use SLF4J.

Levels:

INFO

Business events.

DEBUG

Debugging details.

WARN

Recoverable issues.

ERROR

Failures.

Structured Logging

Include:

- Correlation Id
- Transaction Id
- Request Id

Example:

```
log.info("Order created. orderId={}, customerId={}",  
        orderId, customerId);
```

Never Log

- Passwords
- Tokens
- PAN
- Aadhaar
- PII information

12. Code Complexity

Complexity Threshold

Metric	Recommended
Method Length	< 30 lines
Cyclomatic Complexity	< 10
Nesting Depth	< 3
Class Length	< 300 lines

Reduce Complexity By

- Guard clauses
- Strategy pattern
- Extract methods
- Early returns

13. Java Best Practices

Use Optional Properly

Good:

```
Optional<User> user;
```

Avoid:

```
Optional<List<User>>  
Optional<String> field;
```

Prefer Immutable Objects

Use:

```
final
```

wherever possible.

Use Streams Judiciously

Use streams for readability.

Avoid overly complex streams.

Bad:

```
list.stream()  
    .filter()  
    .map()  
    .flatMap()  
    .collect();
```

when simple loops improve readability.

Prefer Enums Over Constants

Bad:

```
public static final String ACTIVE = "ACTIVE";
```

Good:

```
public enum Status {  
    ACTIVE,  
    INACTIVE  
}
```

14. Performance and Optimization

Database

Avoid N+1 queries

Use:

```
JOIN FETCH  
@EntityGraph
```

Pagination Mandatory

Never fetch unbounded data.

Batch Processing

Use:

```
saveAll()
```

for bulk operations.

Caching

Use:

```
@Cacheable
```

for frequently accessed data.

Connection Pooling

Use HikariCP.

Tune:

- maxPoolSize
 - minimumIdle
 - connectionTimeout
-

Async Processing

Use:

```
@Async
```

for long-running operations.

15. Security Standards

Authentication

Use:

- OAuth2
 - JWT
 - SSO
-

Authorization

Implement:

Role-based access control

Input Validation

Use:

```
@Valid
@NotBlank
@NotNull
@Pattern
@email
```

Secure Headers

Enable:

```
HSTS
CSP
X-Frame-Options
```

Secrets Management

Never store:

```
passwords
tokens
keys
```

in source code.

Use:

```
Vault
Kubernetes Secrets
Environment Variables
```

16. API Standards

REST Naming

Use nouns.

Good:

```
GET /users
POST /users
DELETE /users/1
```

Bad:

```
/getUsers
/createUser
```

Version APIs

```
/api/v1/users
```

Return Proper Status Codes

Use:

```
200
201
204
400
401
403
404
409
500
```

17. Transaction Management

Use:

```
@Transactional
```

only in service layer.

Keep transactions short.

Avoid nested transactions.

18. Testing Standards

Unit Testing

Framework:

- JUnit 5
- Mockito

Coverage target:

```
80%+
```

Test:

- Positive scenarios
 - Negative scenarios
 - Edge cases
-

Naming Convention

```
shouldCreateUserSuccessfully()  
shouldThrowExceptionWhenUserNotFound()
```

Integration Testing

Use:

```
@SpringBootTest  
@Testcontainers
```

19. Code Review Checklist

Before merge verify:

- Naming standards followed
 - No duplicate code
 - SOLID principles followed
 - Tests added
 - No code smells
 - No hardcoded values
 - Exceptions handled
 - Security reviewed
 - Performance reviewed
 - Logging added
 - Documentation updated
-

20. Static Code Analysis

Mandatory tools:

- SonarQube
- Checkstyle
- PMD
- SpotBugs

Quality Gates:

```
Coverage >= 80%  
Critical Issues = 0  
Blocker Issues = 0  
Duplication < 3%
```

21. Observability Standards

Applications must expose:

- Health endpoints
- Metrics
- Traces
- Logs

Use:

```
Spring Boot Actuator  
Micrometer  
Prometheus  
Grafana  
ELK  
OpenTelemetry
```

22. Configuration Management

Externalize all configurations.

Use:

Profiles:

```
dev  
qa  
uat  
prod
```

Never hardcode environment-specific values.

23. CI/CD Requirements

Pipeline should include:

1. Compile
 2. Unit Tests
 3. Static Analysis
 4. Security Scan
 5. Coverage Validation
 6. Artifact Generation
 7. Deployment
 8. Smoke Test
-

24. Dependency Management

Guidelines:

- Remove unused dependencies.
- Upgrade dependencies regularly.
- Monitor CVEs.

Recommended tools:

- OWASP Dependency Check
 - Dependabot
 - Snyk
-

25. Definition of Done

A story is considered complete only when:

- Code reviewed
 - Unit tests passed
 - Sonar quality gate passed
 - Security scan passed
 - Documentation updated
 - Logs added
 - Monitoring added
 - No critical defects open
 - Deployed successfully in lower environment
-

26. Core Engineering Principles

1. Simplicity over complexity.
2. Readability over cleverness.
3. Composition over inheritance.
4. Fail fast.
5. Secure by default.
6. Automate everything.
7. Continuously refactor.
8. Measure before optimizing.
9. Build observable systems.
10. Leave code better than you found it.